

Windows Registry

The Windows Registry is a hierarchical database storing configuration settings for the OS, hardware, installed software, and per-user preferences. For your path, it matters in three ways: **persistence mechanisms** (malware/attackers love the registry), **privilege escalation** (misconfigured permissions), and **forensics/SOC investigation** (registry keys are a goldmine of evidence).

1. What is the Registry, Really?

Think of it as a giant, structured settings database — replacing the old INI-file-per-application approach. Every setting (screen resolution, installed programs, startup items, security policies) lives here as a key-value structure, similar conceptually to a nested folder tree where "folders" are keys and "files" are values.

2. The Five Root Hives (Top-Level Keys)

Hive	Abbreviation	Purpose
HKEY_LOCAL_MACHINE	HKLM	System-wide settings (hardware, installed software, services) — affects all users
HKEY_CURRENT_USER	HKCU	Settings for the currently logged-in user only
HKEY_USERS	HKU	Contains HKCU-equivalent data for all loaded user profiles on the machine
HKEY_CLASSES_ROOT	HKCR	File associations, COM/OLE object registration (merged view of HKLM + HKCU class data)
HKEY_CURRENT_CONFIG	HKCC	Current hardware profile info (rarely used directly today)

The two you'll touch 90% of the time: **HKLM** (system/machine-wide — needs admin to modify) and **HKCU** (per-user — no admin needed, which is exactly why it's a favorite for user-level persistence/malware).

3. Registry Structure — Keys, Subkeys, Values

```

HKEY_LOCAL_MACHINE
├── SOFTWARE
│   ├── Microsoft
│   │   ├── Windows
│   │   │   ├── CurrentVersion
│   │   │   │   ├── Run ← this is a "key"
│   │   │   │   │   ├── OneDrive = "C:\Users\0mkar\...\OneDrive.exe" ← "value
│   │   │   │   │   │   name" = "data"
│   │   │   │   │   └── SecurityHealth = "%windir%\...\SecurityHealthSystray.ex
│   │   │   │   │       e"

```

- **Key** — like a folder (e.g., `Run`)
- **Value name** — like a filename inside that folder (e.g., `OneDrive`)
- **Value data** — the actual content stored (e.g., the file path)
- **Value type** — the data type (see below)

4. Registry Value Types

Type	Symbol	Meaning
<code>REG_SZ</code>	String	A simple text string
<code>REG_DWORD</code>	32-bit number	Common for flags/settings (0 or 1, enable/disable)
<code>REG_QWORD</code>	64-bit number	Larger numeric values
<code>REG_BINARY</code>	Raw binary	Raw byte data
<code>REG_MULTI_SZ</code>	Multi-string	Array of strings
<code>REG_EXPAND_SZ</code>	Expandable string	Contains environment variables like <code>%windir%</code> that get resolved at runtime

5. Viewing/Editing the Registry — GUI

`regedit`

Opens the **Registry Editor GUI** — tree view on the left (hives → keys → subkeys), values on the right. Good for manual browsing, but for scripting/automation (your actual use case), CLI tools matter more.

6. Command Line — `reg` command

```
:: Query a key (view its values)
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

:: Query recursively
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s

:: Add a new value
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v MyApp /t REG_SZ /d "C:\myapp.exe" /f

:: Delete a value
reg delete "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v MyApp /f

:: Export a key to a .reg file (backup)
reg export "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" run_backup.reg

:: Import a .reg file
reg import run_backup.reg

:: Compare two keys
reg compare "HKLM\SOFTWARE\App1" "HKLM\SOFTWARE\App2"
```

Flags explained:

- `/v` — value name
- `/t` — value type (REG_SZ, REG_DWORD, etc.)
- `/d` — the data
- `/f` — force (skip confirmation prompt)

7. PowerShell — The Registry as a "Drive"

This is genuinely elegant — PowerShell treats the registry like a filesystem, so you can navigate it with the same cmdlets you'd use for files:

```

# Navigate like a filesystem
Set-Location HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
Get-ChildItem

# Read a value
Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"

# Read a specific value
(Get-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run").OneDrive

# Create/set a value
New-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" -Name "MyApp" -Value "C:\myapp.exe" -PropertyType String

# Modify an existing value
Set-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" -Name "MyApp" -Value "C:\newpath.exe"

# Delete a value
Remove-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" -Name "MyApp"

# Create a whole new key
New-Item -Path "HKCU:\SOFTWARE\MyNewKey"

```

8. Key Registry Locations Worth Memorizing

a) Autorun / Persistence Locations

```

HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce

```

Anything listed here launches automatically at logon/startup. This is **the** classic persistence mechanism — both for legitimate software (like OneDrive, antivirus) and malware. First thing checked in almost any Windows incident response or malware triage.

```
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"  
reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
```

b) Services

```
HKLM\SYSTEM\CurrentControlSet\Services
```

Every installed service, its binary path, start type, and running account. Attackers modifying a service's `ImagePath` here (if they have write access) is a classic privesc technique — same concept as the "writable service binary" attack, but via registry instead of file replacement.

```
Get-ItemProperty "HKLM:\SYSTEM\CurrentControlSet\Services\Spooler"
```

c) Installed Software

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall
```

Lists every installed application — this is literally what "Add/Remove Programs" reads from. Useful for asset inventory / recon during an internal pentest.

d) AlwaysInstallElevated (privesc misconfiguration)

```
HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer → AlwaysInstallElevated  
HKCU\SOFTWARE\Policies
```